



Blazor Essencial

Ciclo de vida dos componentes





Ciclo de vida dos componentes

O **ciclo de vida** de um componente Blazor é a **sequência de eventos** executados desde a criação do componente até a sua remoção da interface do usuário (UI)

Para permitir reagir a cada um desses **eventos**, o Blazor oferece **métodos virtuais** (como ***OnInitializedAsync, OnParametersSetAsync, OnAfterRenderAsync, etc.***)

Ele são **herdados** da classe base **ComponentBase**, que fornece os métodos do ciclo de vida.

Estes métodos:

- Permitem executar *lógica personalizada* em momentos específicos.
- São essenciais para *gerenciar hierarquias, modos de renderização, tarefas assíncronas e desempenho.*

Os métodos podem ser implementados usando as palavras-chave **protected** (com exceção de ***SetParametersAsync***) e **override** para estender o comportamento da classe base. (que fornece métodos virtuais)



Ciclo de vida dos componentes : ordem de execução (Eventos)

Ordem

SetParametersAsync

Intercepta e processa os parâmetros do componente
Chama **OnInitialized/Async** e **OnParametersSet/Async**

1º Sempre

OnInitialized
OnInitializedAsync

Inicialização única do componente após a definição dos parâmetros. (*Prepara o componente*)

2º Apenas na primeira renderização

OnParametersSet
OnParametersSetAsync

Executado uma vez após **OnInitialized/Async**
Toda vez após os parâmetros serem atualizados

3º Sempre que o Pai renderiza

ShouldRender (*otimização*)

Determina se o componente deve ser renderizado

4º Após **OnParametersSet** ou **StateHasChanged**

Renderização (Construção da Árvore de Renderização)

5º Se **ShouldRender** for true

OnAfterRender
OnAfterRenderAsync

Lógica pós-renderização.
Ideal para interação JS e obter referências a elementos

6ª (Sempre após a renderização)

Dispose
IAsyncDisposable

Liberação de recursos ao descartar o componente

Final (Remove o componente)



Ciclo de vida dos componentes : ordem de execução (Eventos)

1. Ciclo de Vida de CRIAÇÃO (A Primeira Vez)

SetParametersAsync é chamado (*recebendo os parâmetros iniciais*).

OnInitialized() / OnInitializedAsync() são chamados na inicialização.

OnParametersSet() / OnParametersSetAsync() são chamados.

O componente é renderizado (a UI é desenhada na tela).

OnAfterRender() / OnAfterRenderAsync() são chamados (com **firstRender** sendo passado como **true**).

ShouldRender não é chamado

2. Ciclo de Vida de ATUALIZAÇÃO (Das Vezes Seguintes)

Evento Ocorre (*Ex: clique de botão*).

SetParametersAsync é chamado (se os parâmetros mudaram).

OnParametersSet() / OnParametersSetAsync() são chamados (*se os parâmetros mudaram*).

ShouldRender() é chamado.

Se **true**:

O componente é re-renderizado.

OnAfterRender() / OnAfterRenderAsync() são chamados (com **firstRender** sendo passado como false).

Se **false**:

O processo para.

Objetivo : Interceptar e processar parâmetros antes de defini-los nas propriedades do componente.

Primeiro método chamado no ciclo de vida de um componente Blazor.

Permite manipular parâmetros antes do evento **OnParametersSetAsync**, oferecendo uma oportunidade para **validação** ou transformação.

Raramente é **sobrescrito**, sendo usado para cenários avançados (*ex.: validação de parâmetros ou inicialização condicional*).(A Implementação Padrão Já Faz o Trabalho Essencial)

Quando o componente é renderizado (*seja por rota ou por um componente pai*), o Blazor chama **SetParametersAsync** e passa **TODOS** os parâmetros na struct **ParameterView**, independente de onde vieram!



SetParametersAsync : Componente Pai que passa os parâmetros

```
@page "/pai"
```

```
@rendermode InteractiveServer
```

Componente Pai

```
<button @onclick='()' => MudarParametros(1, "João")' class="btn btn-primary">ID 1, Nome João</button>  
<button @onclick='()' => MudarParametros(2, "Maria")' class="btn btn-primary">ID 2, Nome Maria</button>  
<button @onclick='()' => MudarParametros(0, " ")' class="btn btn-warning">IDs Inválidos</button>
```

```
<Filho ItemId="@idAtual" Nome="@nomeAtual" />
```

```
@code {  
    private int idAtual = 1;  
    private string nomeAtual = "João";  
  
    private void MudarParametros(int novold, string novoNome)  
    {  
        idAtual = novold;  
        nomeAtual = novoNome;  
    }  
}
```

Componente Pai passa dois parâmetros para o componente Filho



SetParametersAsync : Componente filho que recebe os parâmetros

<h4>Filho (Recebe o parâmetro)</h4>

Componente Filho

```
@if (ItemId <= 0 || string.IsNullOrEmpty(Nome))
{
    <p style="color: red;font-size: 30px">✗ Dados inválidos!</p>
    <p style="font-size: 30px">ID: @ItemId, Nome: '@Nome'</p>
}
else
{
    <p style="color: green;font-size: 30px">✓ Dados válidos!</p>
    <p style="font-size: 30px">ID: @ItemId, Nome: @Nome</p>
}

@code {
    [Parameter] public int ItemId { get; set; }
    [Parameter] public string? Nome { get; set; }

    public override async Task SetParametersAsync(ParameterView parameters)
    {
        Console.WriteLine("SetParametersAsync chamado!");
        parameters.SetParameterProperties(this);
        Console.WriteLine($"ItemId recebido: {ItemId}");
        await base.SetParametersAsync(ParameterView.Empty);
        Console.WriteLine("Processamento completo!");
    }
}
```

ESTA PARTE SÓ RENDERIZA DEPOIS do **SetParametersAsync**

A sobrescrita do método é usada para :

- Validar parâmetros antes do processamento
- Executar lógica customizada quando parâmetros mudam
- Controlar o fluxo de renderização

Esta é a assinatura exata que o Blazor espera.

Pega os valores do **ParameterView** e atribui às propriedades **[Parameter]** do seu componente.

Chama o processamento NORMAL do Blazor (do componente base).

SÓ AGORA o HTML acima será renderizado!


```
public override async Task SetParametersAsync(ParameterView parameters)
{
    // PASSO 1: O desenvolvedor faz a atribuição MANUAL
    parameters.SetParameterProperties(this);

    //VALIDAÇÃO DE PARÂMETROS É FEITA AQUI

    // PASSO 2: O desenvolvedor continua o ciclo de vida
    await base.SetParametersAsync(ParameterView.Empty);
}
```

Uma vez que você sobrescreve **SetParametersAsync**, a implementação padrão do **ComponentBase** não é mais executada. Isso significa que **OnInitializedAsync** e **OnParametersSetAsync** não serão mais chamados automaticamente. (Por isso chamamos *base.SetParametersAsync(...)*)

@ OnInitialized / OnInitializedAsync

Objetivo: Executar lógica de inicialização única do componente.

Chamado uma vez após **SetParametersAsync** e antes de **OnParametersSetAsync**

É útil para configurações ou inicializações que *não dependem de parâmetros*

OnInitialized: Para lógica *síncrona*, usada em casos simples.

OnInitializedAsync: Para operações *assíncronas*, como chamadas a APIs ou tarefas que requerem atraso, ideal para carregar dados iniciais.

Exemplo: Criar um componente **Noticias.razor** que usa **OnInitializedAsync** para simular o carregamento de uma lista de notícias e exibir o número de itens na UI após um pequeno atraso. Isso permite demonstrar o comportamento assíncrono em tempo real.

@ OnParametersSet / OnParametersSetAsync

Objetivo: Reagir a mudanças nos parâmetros do componente sendo o ponto ideal para executar lógica assíncrona, como o carregamento de dados de APIs ou bancos, que dependem dos novos valores atribuídos às propriedades **[Parameter]**.

Chamado após o método **OnInitializedAsync** e sempre que os parâmetros mudarem.

Pode ser acionado mesmo *com parâmetros iguais*, se o componente pai for renderizado novamente

Essencial para reagir a *mudanças dinâmicas de parâmetros*, como filtros ou seleções do usuário, mas pode ser otimizado com **ShouldRender**

Exemplo: Vamos criar um componente **PerfilDoUsuario.razor** que recebe um **parâmetro pela URL**. Assim, podemos ver o método ser executado quando a página carrega e, mais importante, quando o *parâmetro na URL muda*.

Objetivo: Determina se o componente deve ser renderizado.

Retorna **true** para renderizar ou **false** para evitar renderização desnecessária.

Usado para otimizar desempenho em atualizações frequentes.

Sobrescrevê-lo pode reduzir o uso de recursos, mas deve ser usado com cuidado para não ocultar mudanças visíveis na UI.

Exemplo: Vamos criar um componente **ContadorInteligente.razor** que vai usar o método **ShouldRender** para exibir na tela somente os números pares

@ OnAfterRender / OnAfterRenderAsync

Objetivo: Executar qualquer lógica que dependa do conteúdo HTML ter sido totalmente renderizado e aplicado ao **Document Object Model (DOM)** real do navegador.

São chamados **após** a interface do usuário ter sido exibida na tela, tornando-os o local ideal para interagir com **JavaScript** ou bibliotecas de terceiros (*como mapas ou charts*) que precisam manipular elementos do DOM.

Recebe um argumento bool **firstRender** que permite identificar se é a primeira renderização.

Exemplo: Criar um componente **Alerta.razor** que inicializa um alerta **JavaScript** após a primeira renderização. (**firstRender**)

@ Dispose / IDisposable

Objetivo: Liberação de recursos ao descartar o componente.

Chamado quando o componente é removido da UI, permitindo a liberação de recursos não gerenciados (*ex.: conexões, timers, etc.*).

Dispose: Para liberação **síncrona** de recursos.

IDisposable: Para liberação **assíncrona**, como cancelar tarefas em segundo plano.

Crucial para evitar *memory leaks*(*vazamento de memória*) em componentes que usam recursos como **timers ou streams**, especialmente em aplicações de longa execução.

Exemplo: Criar um componente **Cronometro.razor** que atualiza a tela a cada segundo. O problema é: se sairmos da página, o cronômetro continuará rodando em segundo plano para sempre, causando um vazamento de memória. O **Dispose** é a solução para "**desligar**" o cronômetro quando não precisamos mais dele.